SECURE WEB-BASED BANKING SYSTEM

Database Design & Data Integrity

Course: SCMP 401 – Software Engineering Studio

Milestone: 1

Prepared by: Akar Walid – Database Designer

Project Team

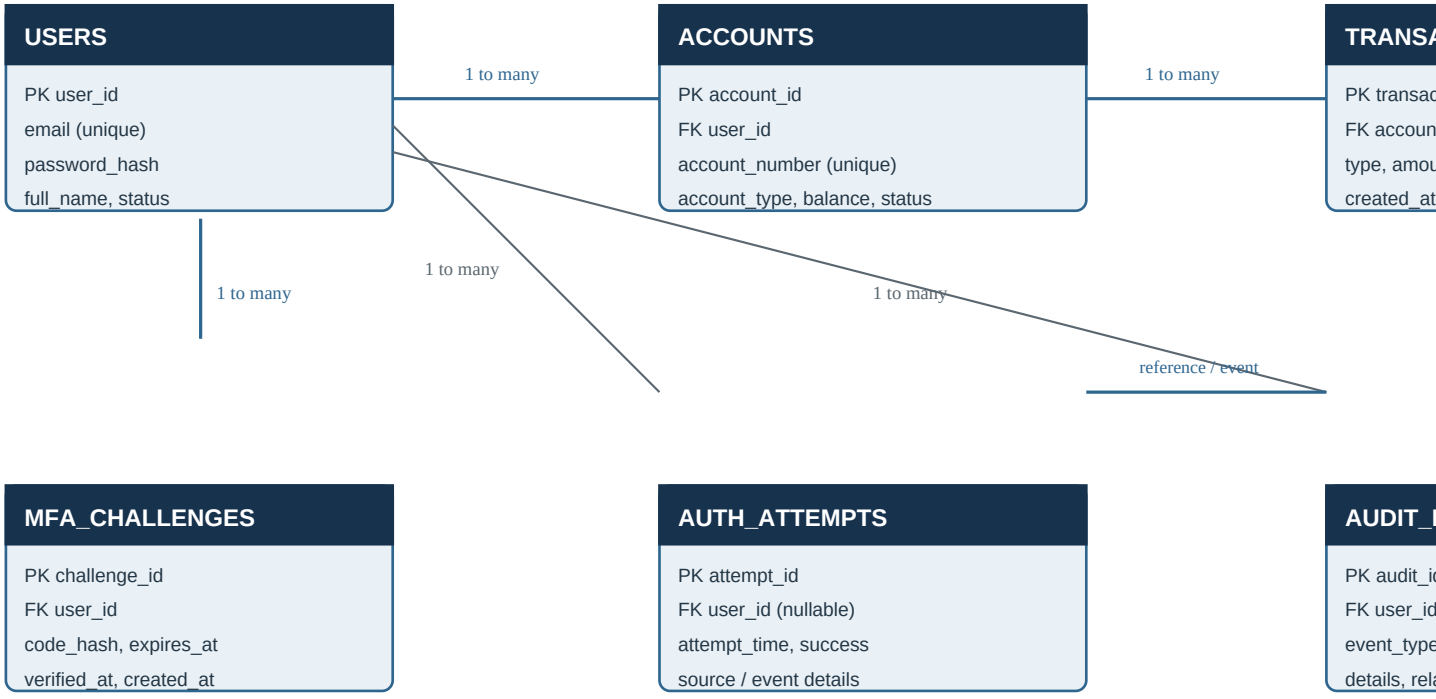
Team Member	Assigned Role
Shakar Tahsin	Product Owner
Shadyar Faruq	Backend Developer
Akar Walid	Database Designer
Akar Omer	UI/UX Designer
Awder Muhamed	System Architecture & Security
Muhamad Jihan	Scrum Master

Purpose: This document proposes the database structure needed to support customer authentication, bank accounts, deposits, withdrawals, and audit records. It is a logical design for team review and implementation.

1. Database Design Overview

The database must store customer information, account details, transaction history, authentication events, and audit records. The design separates these responsibilities into related tables so that data can be maintained consistently and accessed through defined relationships.

2. Proposed Entity Relationship Diagram (ERD)



Relationship summary

- One user can own many accounts; each account belongs to one user.
- One account can have many transactions; each transaction belongs to one account.
- One user can have many MFA challenges and authentication attempts.
- Audit records may reference the user who performed an action and, where applicable, a related transaction or account.

The diagram is a logical proposal. The team should confirm final field types, optional relationships, and implementation details with the backend and architecture members.

3. Proposed Tables and Main Fields

USERS

Field	Key / Rule	Purpose
user_id	Primary key	Unique customer identifier
full_name	Required	Customer's name
email	Unique, required	Login/contact identifier
password_hash	Required	BCrypt password hash; never plaintext
status	Required	For example: ACTIVE or LOCKED
created_at	Required	Account creation timestamp

ACCOUNTS

Field	Key / Rule	Purpose
account_id	Primary key	Unique bank account record
user_id	Foreign key → USERS	Owner of the account
account_number	Unique, required	Customer-facing account number
account_type	Required	CURRENT or SAVINGS
balance	Decimal, required	Current stored balance
status	Required	For example: ACTIVE or CLOSED
overdraft_limit	Decimal	Current Account rule; template specifies $500
minimum_balance	Decimal	Savings Account rule; template specifies $100
created_at	Required	Account creation timestamp

TRANSACTIONS

Field	Key / Rule	Purpose
transaction_id	Primary key	Unique transaction identifier
account_id	Foreign key → ACCOUNTS	Account affected
transaction_type	Required	DEPOSIT, WITHDRAWAL, or INTEREST
amount	Positive decimal	Transaction amount
status	Required	PENDING, COMPLETED, or FAILED
created_at	Required	Transaction timestamp
reference	Unique where used	Reference for tracking/reconciliation

MFA_CHALLENGES

Field	Key / Rule	Purpose
challenge_id	Primary key	Unique challenge identifier
user_id	Foreign key → USERS	User receiving the challenge
code_hash	Required	Store a protected representation, not a plain code
expires_at	Required	Challenge expiration time
verified_at	Nullable	Set when verification succeeds
created_at	Required	Challenge creation time

AUTH_ATTEMPTS

Field	Key / Rule	Purpose
attempt_id	Primary key	Unique authentication event
user_id	Nullable FK → USERS	Known user, if identifiable

Field	Key / Rule	Purpose
attempt_time	Required	Time of attempt
success	Required	Whether authentication succeeded
event_details	Optional	Limited diagnostic/security context

AUDIT_LOGS

Field	Key / Rule	Purpose
audit_id	Primary key	Unique audit event
user_id	Nullable FK → USERS	Actor, if known
event_type	Required	Action recorded
timestamp	Required	When the event occurred
details	Optional	Relevant non-sensitive event information
related_reference	Optional	Related account/transaction reference

4. Data Integrity Rules

- **Primary keys:** every table must have a primary key so each record can be identified uniquely.
- **Foreign keys:** account records must reference an existing user; transaction records must reference an existing account; authentication records must reference a user when the user is known.
- **Uniqueness:** email and account_number must be unique.
- **Valid amounts:** deposits and withdrawals must use positive monetary amounts. Store money using a fixed-precision decimal type, not floating-point.
- **Account types and statuses:** restrict values to defined options such as CURRENT/SAVINGS and ACTIVE/CLOSED.
- **Balance consistency:** update the account balance and insert its transaction record in one database transaction. If either operation fails, roll back both.
- **Withdrawal rules:** validate account status and applicable account rules before committing. Current Account overdraft limit is $500; Savings Account minimum balance is $100, according to the supplied SRS template.
- **Transaction history:** completed transaction records should not be silently overwritten or deleted. Corrections should be represented by a controlled reversal/adjustment record.
- **Audit integrity:** audit records should be append-only and protected from ordinary application-user modification.

5. Security and Privacy Considerations

- Store passwords as BCrypt hashes using the cost factor 12 specified in the SRS template. Never store plaintext passwords.
- Do not store MFA codes in plaintext. Store a protected representation and enforce expiry and one-time verification in the application.
- Restrict database access by application role and grant only the permissions required for each operation.
- Do not place passwords, MFA codes, or other authentication secrets in audit log details.
- Protect network traffic using TLS 1.3 as specified in the project requirements. The assignment announcement also mentions AES-256; confirm with the team whether this is the required method for encryption at rest.

6. Transaction Processing Notes

Deposit: validate the customer session, account ownership, active account status, and amount. Within one database transaction, increase the balance and insert the completed deposit record. Commit only if both succeed.

Withdrawal: validate the customer session, account ownership, active account status, positive amount, and applicable account rules. Within one database transaction, verify the available balance, decrease the balance, and insert the completed withdrawal record. If validation or any database operation fails, make no partial change.

7. Assumptions and Items to Confirm

- The database platform and exact SQL data types have not been specified in this logical design.
- The account number format, currency handling, and whether customers may own multiple accounts should be confirmed by the team.
- The design uses a stored account balance plus transaction history. The backend/database team must agree how the balance is reconciled against the transaction records.
- Interest calculation is specified in the SRS template as 2.5% annually on the minimum daily balance, calculated on the last calendar day of each month. Confirm whether interest is stored as an INTEREST transaction.
- Final table names, columns, constraints, and ERD cardinalities should be reviewed with the Backend Developer and System Architecture & Security member before implementation.

Database Designer contribution: Akar Walid is responsible for refining this logical model into the final ERD and database schema, defining keys and constraints, and coordinating with the backend developer so account balances and transaction records remain consistent.